LLM Interview Question Bank: 7–8-darslar

30 ta savol va javob · Easy, Medium, Advanced · Asadbek

Easy: asosiy tushunchalar (1–10)

1. vocab_size parametri nimani bildiradi va uning qiymati qayerdan olinadi (nimaga bog'langan)?

Qo'shimcha savol: Agar foydalanuvchi model ishlayotgan paytda lug'atda yo'q bo'lgan so'z yoki matn kiritsa, kod xato berib to'xtab qoladimi? Nima uchun?

JAVOB

> vocab_size - unique tokenlar soni. Masalan vocab_size=8000 bo'lsa, unique tokenlar soni 8000 ta bo'ladi va ularning har birining unique id si bo'ladi. vocab_size ni tokenizer ni train qilishdan oldin o'zimiz tanlaymiz: BPE 256 ta baytdan boshlab eng ko'p uchraydigan juftlarni birlashtirib boradi va lug'at 8000 ga yetganda to'xtaydi. Modeldagi vocab_size tokenizer dagi vocab_size bilan bir xil bo'lishi shart (tokenizer.json = run_01.yaml = 8000). Manashu unique tokenlar orqali so'zlar yasaladi.
>
> Agar foydalanuvchi lug'atda yo'q so'zni kiritsa, kod xato bermaydi. So'z lug'atda bor kichikroq tokenlarga bo'linadi, kerak bo'lsa baytlarga bo'linadi. 256 ta bayt lug'atda doim bor, shuning uchun byte-level BPE da `<unk>` token kerak emas.

2. block_size (context size) nima, va u amalda qanday cheklovlarni keltirib chiqaradi?

Qo'shimcha savol: Nima uchun bu loyihada u 256 qilib olingan, sanoatdagi kabi 4096 emas?

JAVOB

> block_size - maximum context size. Masalan block_size = 256 bo'lsa, model bir vaqtda maksimal 256 ta tokenni ko'radi, ya'ni 256 ta token bilan Attention qila oladi. Bu qattiq chegara, chunki position embedding faqat 256 ta pozitsiya uchun bor.
>
> Model bergan javob 256 tokendan oshishi mumkin: generatsiyada model har safar faqat oxirgi 256 ta tokenga qaraydi, oldingilarini esa unutadi. Javob uzunligini `max_tokens` belgilaydi.
>
> block_size katta bo'lsa, model ko'proq contextni tushunadi. Lekin qancha katta bo'lsa, memory va hisoblash shuncha ko'p sarflanadi, va bu o'sish kvadratik: attention har bir tokenni har bir token bilan solishtiradi (T × T). 256 dan 4096 ga o'tilsa, attention hisobi 256 baravar oshadi.
>
> Bu loyihada 256 qilib olingan, chunki: model kichik, GPU cheklangan (noutbuk, Colab T4), uzun context o'rniga ko'proq data ko'rish foydaliroq, va bu o'quv loyihasi.

3. n_embd (bizda d_model) nima, va u kirib kelayotgan tokenlarni qanday ko'rinishga keltiradi?

Qo'shimcha savol: Agar n_embd ni 384 dan 128 ga tushirsangiz, bu modelning ma'noni ifodalash qobiliyatiga qanday ta'sir qiladi?

JAVOB

> n_embd - embedding ning o'lchami, ya'ni har bir token vektorining uzunligi. Bu butun modelning kengligi ham: 384 o'lchamli vektor barcha layer lardan o'tadi.
> token → token id → embedding jadvali (vocab_size × n_embd) → vektor.
>
> 1 ta tokenimizning id si bor, lekin u hech narsani anglatmaydi, shunchaki jadvaldagi qator raqami. Embedding uni vektorga aylantiradi: masalan n_embd=384 bo'lsa, [0.22, -0.15, ..., 0.45] ko'rinishida 384 ta qiymat bo'ladi. Har bir qiymat o'sha tokenning ma'lum bir ma'nosini anglatadi (1 ta son aniq 1 ta ma'no degani emas, ma'no ko'p sonlarning birikmasida bo'ladi). O'xshash ma'noli tokenlarning vektorlari bir-biriga yaqin bo'ladi.
>
> n_embd ni 384 dan 128 ga tushirsak, parametrlar taxminan 9 baravar kamayadi (d^2 ga bog'liq). Model tezroq va kichikroq bo'ladi, lekin ma'nolarni ifodalash uchun joy kamayadi: o'xshash so'zlar aralashib ketadi va model kamroq narsa o'rganadi. 128 / 6 butun son bo'lmagani uchun n_head ni ham o'zgartirish kerak bo'ladi.

4. Model chuqurligi (n_layer) oshganda arxitekturada nima o'zgaradi?

Qo'shimcha savol: 1–2-layer lar o'rganadigan narsalarni 5–6-layer lar o'rganadigan narsalar bilan solishtiring.

JAVOB

> n_layer - bu transformer layer lar soni. Har bir layer da Attention, FFN, 2 ta LayerNorm va 2 ta residual bor, va ular ketma-ket ishlaydi. Har bir layer oldingi layer natijasini olib, uni yanada boyitadi. n_layer oshsa, parametrlar ham chiziqli oshadi (har bir layer ≈ $12 \cdot d^2$), model chuqurroq o'ylay oladi, lekin sekinroq ishlaydi va o'qitish qiyinroq bo'ladi.
>
> Birinchi layer larda (1–2) model oddiyroq bog'lanishlarni o'rganadi: token bo'laklarini so'zga birlashtirish, qo'shni so'zlar, qo'shimchalar, grammatika. Keyingi layer larda (5–6) murakkab bog'lanishlarni o'rgana boshlaydi: gapning ma'nosi, mavzu, uzoqdagi so'zlar o'rtasidagi bog'lanish, va keyingi tokenni bashorat qilish. Bu qat'iy qoida emas, tendensiya.

5. Attention head lar (n_head) nima va nima uchun ular parallel ishlaydi?

Qo'shimcha savol: Nima uchun n_embd n_head ga qoldiqsiz bo'linishi shart? Agar bo'linmasa, PyTorch da nima buziladi?

JAVOB

> n_head - attention head lar soni. n_head = 6 bo'lsa, n_embd ni n_head ga bo'lamiz: 384 / 6 = 64. Demak 6 ta head bo'ladi, har birining o'lchami (head_dim) 64. Bu head lar parallel ishlaydi: har bir head da o'zining 64 o'lchamli Query, Key va Value lari hisoblanadi, va har bir head turli bog'lanishlarga qaraydi (grammatika, ma'no, yaqin so'zlar). Head lar bir-biriga bog'liq bo'lmagani uchun GPU ularni bir vaqtda hisoblaydi. 6 ta head dan chiqqan qiymatlar oxirida birlashtiriladi va W_O orqali aralashtiriladi. 384 → 64, 64, ..., 64 (6 ta) → birlashtirish → W_O → 384.
>
> n_embd n_head ga qoldiqsiz bo'linishi shart, chunki kodda vektor `view(B, T, n_head, head_dim)` bilan bo'linadi. Masalan 384 / 5 = 76.8 bo'ladi, head_dim = 76, 5 × 76 = 380 ≠ 384, va PyTorch `view` da xato beradi. Shuning uchun config da buni oldindan tekshiramiz.

6. Pretraining paytida dropout: 0.1 qanday regularizatsiya (overfitting ga qarshi) ta'sirini beradi?

Qo'shimcha savol: Model foydalanuvchilarga javob berayotganda (inference) dropout yoqilgan bo'lishi kerakmi? Javobingizni asoslang.

JAVOB

> dropout - regularizatsiya hisoblanadi, ya'ni overfitting ning oldini oladi. 0.1 qiymat har bir qadamda neyronlarning 10% ining chiqishini random 0 ga aylantiradi (weight lar o'chirilmaydi), qolgan neyronlarning chiqishi esa 1/0.9 ga ko'paytiriladi, shunda o'rtacha qiymat o'zgarmaydi. Model aniq bir neyronga bog'lanib qolgan bo'lishi mumkin, shuning uchun neyronlar random tashlab yuborilganda bilim qolganlariga tarqatiladi.
>
> Bu faqat training da ishlaydi, inference da dropout dan foydalanilmaydi (`model.eval()` uni o'chiradi). Chunki training tugaganda biz parametrlarni olgan bo'lamiz, va ularning har biri foydali. Model javob berayotganda ularni tashlab yuborish nimaga kerak? Dropout ning vazifasi overfitting ni oldini olish, javob berayotganda esa overfitting degan tushuncha yo'q. Agar inference da dropout yoqilib qolsa, javoblar shovqinli va yomonroq bo'ladi, va bir xil prompt ga har safar boshqacha natija chiqadi.

7. Feed-Forward Network (FFN) qatlamining asosiy vazifasi nima?

Qo'shimcha savol: Nima uchun FFN ning ichki qatlami avval ffn_mult (bizda mlp_ratio) orqali 4 baravar kengaytiriladi, keyin yana asl o'lchamga siqiladi?

JAVOB

> FFN ning vazifasi har bir token bilan alohida-alohida ishlash: tokenlar bir-biri bilan gaplashmaydi (bu ish Attention da bajariladi). FFN modelning bilim ombori: Attention yig'gan kontekstga qarab, tokenga modelning bilimini qo'shadi, ya'ni token qanday ma'noni anglatishini aniqlashtiradi. Modeldagi parametrlarning katta qismi ($12 \cdot d^2$ dan $8 \cdot d^2$) FFN da.
>
> ffn_mult ning 4 baravar kengaytirishini shunday qiyoslasak bo'ladi: bizda kichikroq quti bor, ichi to'la sharlar, va sharlarni saralashimiz kerak. Bu holatda saralash qiyin bo'ladi. Agar qutimiz kattaroq bo'lsa (4 baravar), sharlarni saralash ancha oson kechadi. FFN da ham shunday: 384 → 1536 da 1536 ta neyron (detektor) tokenni tekshiradi, GELU faqat mos kelganlarini qoldiradi, va tokenning ma'nosi aniqroq aniqlanadi. Keyin natija yana 384 ga siqiladi, chunki residual stream va keyingi layer 384 o'lchamli vektor kutadi.

8. tie_weights: true nimani anglatadi va u qanday tuzilgan?

Qo'shimcha savol: Bu tanlov modeldan nechta parametrni olib tashlaydi?

JAVOB

> tie_weights: true weight larni bog'lash degani. Modelga so'z kiradi → token → token id → embedding (E matritsasidan qator olinadi) → transformer → chiqishda vektor h. Foydalanuvchiga natijani token ko'rinishida ko'rsatish uchun h o'sha E matritsasi bilan solishtiriladi (`logits = h @ E.T`): barcha 8000 ta token ball oladi, va h ga eng yaqin vektorli token tanlanadi. Ya'ni kirishda token → vektor, chiqishda esa vektor → token uchun bitta matritsa ishlatiladi.
> Masalan "Hi" → [0.244, 0.344, ..., 0.555]. Agar model chiqishda shunga yaqin vektor hosil qilsa, "Hi" eng katta ballni oladi.
>
> Bu bitta matritsani saqlagani uchun $8000 \times 384 = 3.07M$ parametr tejaladi. Bundan tashqari, matritsa ham kirish, ham chiqish tomonidan o'rganadi, bu esa ayniqsa kam uchraydigan token lar uchun foydali.

9. Training ning 0-qadamida model holatini Random-init loss check orqali qanday baholaysiz?

Qo'shimcha savol: Nima uchun 8000 ta tokenli model uchun boshlang'ich loss aynan taxminan 8.98 bo'lishi kerak?

JAVOB

> Random-init loss check - training boshlanishidan oldin, o'qitilmagan model bilan loss ni hisoblab ko'rish. Model hali hech narsa bilmaydi, shuning uchun weight lar to'g'ri initsializatsiya qilingan bo'lsa (kichik, std 0.02), u 8000 ta tokenning hammasiga deyarli bir xil ehtimol beradi: har biriga 1/8000.
>
> Loss = −log(to'g'ri tokenning ehtimoli) = −log(1/8000) = ln(8000) ≈ 8.98. Shuning uchun boshlang'ich loss taxminan 8.98 bo'lishi kerak.
>
> Agar loss ancha baland chiqsa (masalan, 15), demak model hech narsa bilmasa ham ishonch bilan noto'g'ri javob beryapti: init yoki logits da xato bor. Agar ancha past chiqsa, demak model javobni ko'rib turibdi: mask yoki x/y siljishida xato bor. Bu tekshiruv arzon, lekin GPU vaqti sarflanishidan oldin ko'p xatoni ushlaydi.

10. generate() funksiyasi ichidagi Greedy Decoding nima?

Qo'shimcha savol: Greedy generatsiyaning asosiy kamchiligi nima, va production da u qachon afzal ko'riladi?

JAVOB

> Greedy decoding - model har bir qadamda keyingi token uchun 8000 ta ehtimol hisoblaydi va shulardan eng kattasini tanlaydi (argmax). Keyin shu tokenni matnga qo'shib, yana keyingisini tanlaydi. Bir xil prompt ga doim aynan bir xil javob chiqadi. Sampling da `temperature = 0` qilinsa, u greedy bilan bir xil ishlaydi.
>
> Kamchiligi: har bir qadamda eng yaxshisini tanlash butun matn uchun eng yaxshisi degani emas. Model ko'pincha bir xil so'zlarni takrorlab, siklga tushib qoladi, va matn zerikarli bo'ladi. Shuning uchun matn yozishda sampling (temperature, top-k, top-p) ishlatiladi.
>
> Production da greedy to'g'ri javob bitta bo'lgan joylarda afzal: kod, matematika, ma'lumot ajratib olish, tasniflash, va natija har doim bir xil bo'lishi kerak bo'lgan testlarda.

Medium: arxitektura va matematik mantiq (11–20)

11. Causal Self-Attention ni klassik Global Attention bilan bitta gap misolida solishtiring.

Qo'shimcha savol: Agar pretraining paytida causal mask bo'lmasa, loss grafigi qanday o'zgaradi va nima uchun bu muammo?

JAVOB

Causal Self-Attention (yaratish) - har bir token faqat o'zidan oldingi so'zlarni ko'ra oladi, keyingilari mask bilan yopiladi. Model shu orqali keyingi tokenni taxmin qiladi.
Global Attention (tushunish) - mask yo'q, har bir token o'zidan oldingi va keyingi so'zlarni ko'ra oladi. BERT kabi modellar undan matnni tushunish uchun foydalanadi, masalan, o'rtada tushib qolgan so'zni topish uchun.

Masalan:

- "I love ..." → Causal → "AI" → "I love AI"
- "I ... AI" → Global (BERT) → "love" → "I love AI"

Causal bilan matn generate qila olamiz (GPT, Llama). Global esa matnni tushunish uchun: tasniflash, qidiruv embedding lari, rasmlar (ViT).

Agar pretraining da causal mask bo'lmasa, loss juda tez 0 ga yaqin kamayib ketadi va training zo'r ko'rinadi. Lekin aslida model keyingi tokenni predict qilish o'rniga javobni ko'rib oladi va ko'chiradi. Generatsiya paytida esa keyingi token hali yo'q, shuning uchun model ma'nosiz matn yozadi. Val loss ham past chiqadi, shuning uchun bu xatoni loss dan emas, Causality test dan bilib olamiz.

12. Nima uchun model parametrlari soni xohishga qarab emas, balki mavjud token byudjetiga (data hajmiga) qarab tanlanishi kerak?

Qo'shimcha savol: Chinchilla qoidasiga ko'ra (bir parametrga ≈20 token), 100M tokenli korpus uchun optimal model hajmi qancha?

JAVOB

Model parametrlari soni transformer layer lar soni va embedding o'lchamiga bog'liq (≈ $12 \cdot n_layer \cdot d^2$). Transformer layer qancha ko'p bo'lsa, model oddiy bog'lanishlardan murakkabroq bog'lanishlargacha o'rgana oladi. Embedding o'lchami katta bo'lsa, bitta tokenning ma'nosini kengroq saqlay olamiz. Lekin model hajmi data hajmiga (token byudjetiga) mos bo'lishi kerak:

- model katta, data kam bo'lsa, model chala o'qitiladi yoki datani yodlab oladi, va hisoblash kuchi bekorga ketadi;
- model kichik, data ko'p bo'lsa, model hamma bilimni sig'dira olmaydi.

Chinchilla qoidasi: har bir parametrga taxminan 20 ta token kerak ($D \approx 20 \cdot N$). Demak 100M token uchun optimal model: $N = 100M / 20 = 5M$ parametr.

13. Nima uchun network layer lari nol bilan emas, balki standart og'ishi 0.02 bo'lgan tasodifiy sonlar bilan initsializatsiya qilinadi?

Qo'shimcha savol: Agar 0-qadamda barcha Linear qatlamlarning weight lari aynan 0 qilib qo'yilsa, backpropagation da nima bo'ladi?

JAVOB

> Weight lar nol bilan initsializatsiya qilinmaydi, chunki hamma weight 0 bo'lsa, bitta layer dagi barcha neyronlar bir xil natija chiqaradi, bir xil gradient oladi va bir xil yangilanadi. Ular abadiy bir xil bo'lib qoladi, ya'ni 1536 ta neyron bitta neyrondek ishlaydi. Bundan tashqari, gradient har bir layer da weight ga ko'payib o'tadi, va weight 0 bo'lsa, gradient ham 0 bo'lib, backpropagation da signal o'tmaydi: model o'rganmaydi.
>
> Shuning uchun weight lar kichik tasodifiy sonlar bilan initsializatsiya qilinadi (std 0.02). Tasodifiylik har bir neyronga boshqacha boshlang'ich nuqta beradi (simmetriya buziladi), kichiklik esa sonlarning layer dan layer ga o'sib ketishiga yo'l qo'ymaydi. Natijada model 0-qadamda 8000 ta tokenga deyarli teng ehtimol beradi, va boshlang'ich loss ≈ 8.98 bo'ladi.

14. Chuqur network larda Pre-Norm (LayerNorm ni Attention va FFN dan oldin qo'yish) qanday afzalliklarga ega?

Qo'shimcha savol: Post-Norm 32 tadan ortiq layer li modellarda qanday muammo keltirib chiqaradi?

JAVOB

> Pre-Norm da forward shunday ishlaydi: avval normalizatsiya, keyin Attention yoki FFN, keyin natija residual ga qo'shiladi: `x = x + Attention(LN(x))`. LayerNorm faqat yon yo'lda turadi, asosiy residual yo'li esa toza qoladi. Shuning uchun backprop da gradient oxirgi layer dan birinchi layer gacha to'g'ridan-to'g'ri, so'nmasdan o'tadi va training barqaror bo'ladi. Oxirida yana bitta LayerNorm (`ln_f`) qo'yiladi.
>
> Post-Norm da `x = LN(x + Attention(x))`, ya'ni LayerNorm asosiy yo'lning ustida turadi. Hisoblash miqdori oshmaydi, lekin gradient har bir LayerNorm dan o'tganda o'zgaradi. 32 dan ortiq layer bo'lsa, bu ta'sir to'planadi va gradient birinchi layer larda so'nib qoladi yoki portlaydi: training beqaror bo'ladi, uzoq warmup va kichik lr kerak bo'ladi. Shuning uchun zamonaviy modellar (GPT-2, Llama) Pre-Norm ishlatadi.

15. Residual connection (+) lar gradient yo'llariga aniq qanday xizmat qiladi?

Qo'shimcha savol: Agar 6 layer li modeldan barcha residual bog'lanishlar olib tashlansa, backward gradient qanday yo'l bosib o'tadi?

JAVOB

> Residual connection - ma'lumot yo'qolmasligini oldini oladi: `y = x + f(x)`, ya'ni Attention yoki FFN eski vektorni almashtirmaydi, faqat unga qo'shadi. Backprop da esa hosila `1 + f'(x)` bo'ladi. Shu "1" tufayli gradient residual yo'li orqali o'zgarmasdan, to'g'ridan-to'g'ri birinchi layer largacha yetib boradi. Agar layer foydasiz bo'lsa, u f(x) ≈ 0 ni o'rganadi va modelni buzmaydi.
>
> Agar residual larni olib tashlasak, gradient 6 ta layer ning har biridan ketma-ket o'tadi va hosilalar ko'payadi: f6' × f5' × ... × f1'. Har biri 0.5 bo'lsa, birinchi layer ga 0.5^6 ≈ 0.016, ya'ni signalning 1.6% i yetadi (vanishing gradient), 1 dan katta bo'lsa esa gradient portlaydi. Natijada birinchi layer lar deyarli o'rganmaydi, chuqur modellarda esa training umuman ishlamaydi.

16. Causality Test language model block ining to'g'ri ishlashini qanday tekshiradi?

Qo'shimcha savol: Agar t pozitsiyadagi kelajak tokenni o'zgartirganda t-1 pozitsiyadagi chiqish (logits) ham o'zgarsa, qaysi qism buzilgan?

JAVOB

> Causality test - model kelajakdagi tokenlarni ko'rmayotganini tekshiradi. Model bir xil matn bilan ikki marta ishga tushiriladi: ikkinchi marta t pozitsiyadagi token o'zgartiriladi. t dan oldingi pozitsiyalarning logits lari aynan bir xil qolishi kerak, chunki ular kelajakni ko'rmasligi kerak. t va undan keyingi pozitsiyalar o'zgarishi mumkin.
>
> Agar t pozitsiyani o'zgartirganda t-1 pozitsiyaning logits lari ham o'zgarsa, demak Attention dagi causal mask buzilgan. Chunki modelda faqat Attention turli pozitsiyalarni aralashtiradi: Embedding, LayerNorm va FFN har bir tokenni alohida ishlaydi. Odatiy xatolar: mask qo'llanmagan, tril o'rniga triu ishlatilgan, mask softmax dan keyin qo'yilgan yoki mask o'lchami noto'g'ri kesilgan.

17. Kurs materiallarida FFN har bir pozitsiyada mustaqil ishlaydi deyiladi. Bu nima uchun muhim?

Qo'shimcha savol: Token lar o'rtasidagi o'zaro ta'sir nuqtai nazaridan Self-Attention block ini FFN block i bilan solishtiring.

JAVOB

Self-Attention tokenlarning bir-biri bilan gaplashishini ta'minlaydi, FFN esa har bir tokenni alohida ishlab, uning qanday ma'noni anglatishini aniqlaydi (modelning bilimini qo'shadi). Attention dynamic: qaysi tokenga qancha qarash har bir gap uchun yangidan hisoblanadi, va token vektori kontekstga qarab o'zgaradi. FFN static: uning weight lari har bir token va har bir gap uchun bir xil. Lekin FFN ning chiqishi kirishga bog'liq, kirishda esa Attention qo'shgan kontekst bor.

FFN ning har bir pozitsiyada mustaqil ishlashi muhim, chunki:

- barcha pozitsiyalar bitta matritsa ko'paytmasi bilan parallel hisoblanadi;
- FFN kelajak tokenlarni ko'rmaydi, shuning uchun causal mask faqat Attention da kerak;
- narxi T ga chiziqli o'sadi (Attention da T^2);
- vazifalar aniq bo'lingan: Attention tashiydi, FFN qayta ishlaydi.

18. Overfit-one-batch tekshiruvining asosiy maqsadi nima?

Qo'shimcha savol: Agar model bitta batch dagi loss ni ham nolga yaqin tushira olmasa, nimani debug qilasiz: data massivlarinimi yoki model parametrlarinimi?

JAVOB

Overfit-one-batch test modelning o'rganish qobiliyatini tekshiradi: bitta kichik batch ni 200 marta o'qitib, loss ni 0 ga yaqin tushira oladimi? Shu orqali butun zanjir ishlayotgani tekshiriladi: forward → loss → backward → optimizer.step. Bitta batch ni yodlash eng oson vazifa, shuning uchun model uni yodlab olsa (overfitting bo'lsa), training ni davom ettirish mumkin.

Agar yodlay olmasa, data ni emas, kodni tekshirish kerak. Chunki model bitta batch ni hatto noto'g'ri data bilan ham yodlay oladi. Tekshiriladigan joylar: backward chaqirilganmi, optimizer.step va zero_grad to'g'ri joydami, lr juda kichik emasmi, parametrlar optimizer ga berilganmi, loss to'g'ri hisoblanyaptimi. Test o'tishi esa data to'g'ri ekanini isbotlamaydi, buning uchun shift va random-init loss tekshiruvlari kerak.

19. Generatsiya sikli ichida nima uchun idx_cond = idx[:, -self.config.block_size:] qatori yozilgan?

Qo'shimcha savol: Agar prompt 300 token uzunlikda bo'lsa, block_size esa 256 bo'lsa, ketma-ketlikning qaysi qismi attention head lar uchun butunlay ko'rinmay qoladi?

JAVOB

> `idx_cond = idx[:, -block_size:]` - generatsiya paytida faqat oxirgi 256 ta tokenni olib, modelga beradi. Generatsiyada idx har qadamda bittaga uzayadi, model esa bir vaqtda 256 tadan ko'p tokenni qabul qila olmaydi: position embedding da faqat 256 ta pozitsiya bor. Bu qator bo'lmasa, 257-tokenda model xato beradi. Shu qator bilan esa oyna surilib boradi: model doim oxirgi 256 ta tokenga qarab yozadi, eskilarini esa unutadi.
>
> Agar prompt 300 token bo'lsa, block_size 256 bo'lsa, 300 − 256 = 44, ya'ni promptning birinchi 44 ta tokeni butunlay ko'rinmay qoladi. Generatsiya davom etgan sari yana ko'proq eski tokenlar oynadan chiqib ketadi. Agar muhim ma'lumot promptning boshida bo'lsa, model uni unutadi.

20. Nima uchun sozlamalar kodga qattiq yozilmasdan, alohida configs/run_01.yaml fayliga chiqarilgan?

Qo'shimcha savol: Bu qoida katta jamoada natijalarni aynan qayta takrorlash (reproducibility) imkonini qanday beradi?

JAVOB

> Sozlamalar kodga yozilmaydi, chunki modelni shu sozlamalar bilan bir marta train qilamiz va natijani olamiz. Keyin qiymatlarni o'zgartirib, yangi qiymatlarni run_02.yaml fayliga yozamiz va yana train qilamiz, va hokazo. Oxirida natijalarni solishtiramiz. Eski config o'zgartirilmaydi, aks holda run_01 natijasi qaysi sozlamalardan chiqqani yo'qoladi. Kod esa o'zgarmaydi, faqat config o'zgaradi, shuning uchun yangi xato kirib qolmaydi.
>
> Jamoada bu reproducibility ni ta'minlaydi: config Git da saqlanadi va unda seed ham bor. Har qanday jamoa a'zosi bir xil commit, bir xil data va `python scripts/train.py --config configs/run_01.yaml` buyrug'i bilan aynan o'sha natijani oladi. Checkpoint ichida ham config saqlanadi, shuning uchun har bir model qanday sozlamalar bilan o'qitilgani doim ma'lum.

Advanced: sanoat standartlari va tizimli fikrlash (21–30)

21. Nima uchun zamonaviy production modellar Multi-Head Attention (MHA) o'rniga Grouped-Query Attention (GQA) dan foydalanadi?

Qo'shimcha savol: Production da javob berayotgan paytdagi KV Cache ning VRAM xarajati nuqtai nazaridan GQA ning afzalliklarini tushuntiring.

JAVOB

> KV cache - model javob yozayotganda oldingi tokenlarning K va V vektorlari GPU xotirasida saqlanadi, shunda ularni har qadamda qaytadan hisoblash shart bo'lmaydi. MHA da har bir Query head ning o'z K va V si bor, shuning uchun KV xotira ko'p ketadi. U context uzunligi va foydalanuvchilar soni bilan birga o'sadi va katta modellarda gigabaytlarga yetadi.
>
> GQA da bir nechta Query head bitta K/V head ni bo'lishadi (masalan, 64 ta Q head va 8 ta KV head). KV xotira 8 baravar kamayadi, sifat esa MHA ga yaqin qoladi (MQA da xotira yanada kam, lekin sifat tushadi, ya'ni GQA o'rtacha yechim). Production da bu ikki narsani beradi: bitta GPU ko'proq foydalanuvchiga yoki uzunroq context ga xizmat qila oladi, va javob tezroq chiqadi, chunki generatsiyada GPU ning ko'p vaqti xotiradan K, V ni o'qishga ketadi.

22. Weight lar bog'langan (tied) bo'lganda num_parameters() metodi model hajmini qanday qilib to'g'ri hisoblaydi?

Qo'shimcha savol: Agar parametrlar sonini bog'langan matritsani ikki marta sanagan holda e'lon qilsangiz, bu model natijalarini baholashni qanday buzadi?

JAVOB

> Weight lar bog'langanda `lm_head.weight = token_embedding.weight` qatori matritsani nusxalamaydi, ikkala nom bitta obyektga ishora qiladi. `num_parameters()` esa `self.parameters()` orqali sanaydi, u har bir obyektni faqat bir marta qaytaradi. Shuning uchun bog'langan matritsa bir marta sanaladi. Agar `state_dict()` orqali sanasak, ikkala kalit ham hisobga olinadi va matritsa ikki marta sanaladi.
>
> Agar parametrlar ikki marta sanab e'lon qilinsa, model vocab_size × d_model parametrga (masalan, 8000 × 384 = 3.07M) kattaroq ko'rinadi. Natijada Chinchilla nisbati (token/parametr) noto'g'ri hisoblanadi, hisoblash xarajati va xotira noto'g'ri baholanadi, va boshqa modellar bilan solishtirganda model o'z hajmiga nisbatan haqiqatdagidan yomonroq ko'rinadi.

23. Learned Positional Embeddings ni RoPE (Rotary Position Embeddings) bilan solishtiring.

Qo'shimcha savol: RoPE ning qanday matematik xususiyatlari unga pretraining dagidan uzunroq context larga moslashish imkonini beradi?

JAVOB

> Learned positional embedding da so'zning turgan o'rni uchun alohida vektor o'rganiladi (jadval 256 × 384) va u kirishda token embedding ga bir marta qo'shiladi. Bu absolyut pozitsiya: model "men 5-o'rindaman" deb biladi. Jadval faqat 256 qatordan iborat, shuning uchun model 256 dan uzun context ni ko'ra olmaydi.
>
> RoPE esa embedding ga hech narsa qo'shmaydi: u har bir layer da Q va K vektorlarini pozitsiyaga qarab burchakka aylantiradi. Q · K ballida faqat burchaklar farqi qoladi, ya'ni token lar orasidagi masofa: qaysi tokenlar bir-biriga yaqin bo'lsa, ular orasidagi burchak farqi ham kichik bo'ladi. Bu nisbiy pozitsiya, va RoPE da parametr yo'q.
>
> RoPE uzunroq context ga moslasha oladi, chunki u jadval emas, formula: istalgan pozitsiya uchun burchakni hisoblash mumkin, va model nisbiy masofalarni o'rganadi, ular esa uzun matnda ham bir xil qoladi. Amalda juda uzun context lar uchun qo'shimcha usullar (position interpolation, YaRN) kerak bo'ladi, lekin ularni RoPE da qo'llash oson.

24. SwiGLU aktivatsiya arxitekturasi oddiy GELU li FFN dan qanday farq qiladi?

Qo'shimcha savol: SwiGLU dagi qo'shimcha linear qatlam qanday xarajat keltiradi? Bu xarajat benchmark dagi yaxshilanishga arziydimi?

JAVOB

> GELU li FFN da 2 ta matritsa bor: x → W1 (384 → 1536) → GELU → W2 (1536 → 384). Har bir neyron o'zi yoqilish yoki o'chishni hal qiladi.
>
> SwiGLU da esa 3 ta matritsa bor: `gate = SiLU(x @ W_gate)`, `up = x @ W_up`, `out = (gate * up) @ W_down`. up ma'lumotni tashiydi, gate esa uning qanchasi o'tishini boshqaradi (darvoza). Bu oddiy o'chirgich emas, balki dimmer kabi ishlaydi, shuning uchun model nozikroq naqshlarni o'rgana oladi.
>
> Qo'shimcha matritsa tufayli parametrlar ko'payib ketmasligi uchun ichki o'lcham 4 baravar o'rniga 8/3 baravar olinadi. Shunda parametrlar soni bir xil qoladi (d_model = 384 da ikkalasi ham 1.18M). Shazeer (2020) bir xil parametr va hisoblash kuchi bilan SwiGLU pastroq loss berishini ko'rsatdi, shuning uchun bu arziydi va Llama, Mistral, Qwen undan foydalanadi. Narxi esa bitta qo'shimcha matritsa ko'paytmasi va biroz ko'proq oraliq xotira.

25. Nima uchun eng katta (frontier) modellarni pretraining qilishda dropout ishlatilmaydi, maktab darajasidagi kichik loyihalarda esa u kerak bo'ladi?

Qo'shimcha savol: Trillionlab token ustida bir epoch bilan o'qitish klassik overfitting muammosini qanday tabiiy ravishda kamaytiradi?

JAVOB

> Frontier modellarda overfitting bo'lish ehtimoli deyarli yo'q, chunki data hajmi juda katta (trillionlab token) va model har bir matnni faqat bir marta ko'radi (bir epoch). Overfitting model bir xil misollarni qayta-qayta ko'rganda paydo bo'ladi. Bir epoch da esa har bir batch yangi bo'ladi, misolni yodlashning foydasi yo'q, chunki u qaytib kelmaydi. Shuning uchun model faqat umumiy qoidalarni o'rganib loss ni kamaytira oladi, va train loss deyarli val loss ga teng bo'ladi. Bunday holatda dropout faqat o'rganishni sekinlashtiradi va GPU vaqtini bekorga sarflaydi, shuning uchun u o'chiriladi.
>
> Kichik loyihalarda esa data kam, model uni bir necha marta ko'radi, va uni yodlab olish juda oson. Shuning uchun dropout kerak.

26. Nima uchun logits PyTorch ning cross-entropy funksiyasiga berilishidan oldin .view(-1, logits.size(-1)) orqali qayta shakllantiriladi?

Qo'shimcha savol: Ko'p o'lchamli tensor ni bitta tekis (flat) tensor ga aylantirish qanday samaradorlik beradi?

JAVOB

> Model chiqishi 3 o'lchamli: logits (B, T, V) = (32, 256, 8000), targets esa (32, 256). PyTorch ning cross_entropy funksiyasi esa oddiy jadval kutadi: input (N, C) va target (N). Shuning uchun `logits.view(-1, 8000)` barcha bashoratlarni bitta ro'yxatga yoyadi: (8192, 8000), `targets.view(-1)` esa (8192). Har bir qator bitta bashorat, va qaysi matn yoki pozitsiyadan kelgani loss uchun muhim emas.
>
> Samaradorligi: view ma'lumotni nusxalamaydi, faqat unga boshqacha qaraydi, shuning uchun deyarli bepul. GPU barcha 8192 ta bashoratni bitta amal bilan parallel hisoblaydi (Python tsikli kerak emas), va natija bitta o'rtacha loss bo'ladi, backward ham bitta. Agar tensor xotirada ketma-ket joylashmagan bo'lsa, view o'rniga reshape ishlatiladi.

27. Agar training ning boshlang'ich loss i 8.98 o'rniga 15.0 chiqsa, kodning qayerida xato bor?

Qo'shimcha savol: Agar boshlang'ich loss aynan 8.98 chiqsa-yu, lekin keyin umuman kamaymasa, data loader pipeline ning qaysi qismi buzilgan bo'lishi mumkin?

JAVOB

> Boshlang'ich loss 8.98 o'rniga 15.0 chiqsa, model hech narsa bilmasa ham ishonch bilan noto'g'ri javob beryapti, ya'ni logits juda katta. Bu initsializatsiya xatosi: `_init_weights` chaqirilmagan yoki std 0.02 o'rniga katta son yozilgan. Masalan, nn.Embedding ning standart init i std 1.0, va weight tying tufayli LM head ham shu katta matritsadan foydalanib, katta logits chiqaradi. Oxirgi LayerNorm unutilgan bo'lsa ham shunday bo'lishi mumkin.
>
> Agar loss 8.98 dan boshlanib, umuman kamaymasa, init to'g'ri, lekin x va y orasida o'rganiladigan bog'lanish yo'q. Data loader da: train.bin noto'g'ri dtype da o'qilgan (uint16 o'rniga boshqa tip, natijada token ID lar tasodifiy bo'ladi), x va y bir-biriga mos emas (y noto'g'ri siljitilgan yoki alohida shuffle qilingan), yoki .bin boshqa tokenizer bilan yaratilgan. Bularni Shift correct tekshiruvi va val.bin dan matnni dekod qilib ko'rish oldindan ushlaydi.

28. Nima uchun generatsiya siklida aynan next_logits = logits[:, -1, :] qatori yozilgan?

Qo'shimcha savol: Faqat oxirgi pozitsiyani ishlatish har bir token ni yozishdagi hisoblash vaqtiga (latency) qanday ta'sir qiladi?

JAVOB

> Model forward qilganda har bir pozitsiya uchun bashorat chiqaradi: logits (B, T, 8000), va har bir pozitsiya o'zidan keyingi tokenni bashorat qiladi. Training da bu foydali, chunki bitta matndan T ta mashq olinadi. Generatsiyada esa bizga faqat matn oxiridan keyingi token kerak, shuning uchun `logits[:, -1, :]` faqat oxirgi pozitsiyaning 8000 ta balini oladi va shulardan keyingi token tanlanadi.
>
> Oddiy generate da (KV cache siz) bu qator vaqtni kam tejaydi: model baribir barcha pozitsiyalarni hisoblaydi, faqat softmax va sampling kichikroq bo'ladi. Haqiqiy tezlanish ikki usuldan keladi: LM head ni faqat oxirgi pozitsiya uchun hisoblash (`lm_head(x[:, [-1], :])`), va KV cache, bunda oldingi tokenlarning K, V lari saqlanadi va har qadamda faqat yangi token hisoblanadi.

29. Standart self-attention ning kvadratik (T^2) xarajatini tushuntiring va context oynasi hajmining amaliy chegarasi nimada ekanini ayting.

Qo'shimcha savol: Context talabi hardware imkoniyatlaridan oshib ketganda xotira va hisoblash o'rtasidagi nisbat qanday o'zgaradi?

JAVOB

> Self-attention da har bir token har bir token bilan solishtiriladi, natijada T × T o'lchamli ballar jadvali hosil bo'ladi. T 2 baravar oshsa, jadval 4 baravar kattalashadi: 256 da 65 ming katak, 4096 da 16.7 million, 128K da 16 milliard. Bu har bir head va layer uchun takrorlanadi, shuning uchun xotira va hisoblash T^2 ga proporsional o'sadi. Amaliy chegarani GPU xotirasi (training da attention jadvali, inference da KV cache) va vaqt belgilaydi.
>
> Context hardware imkoniyatidan oshganda birinchi bo'lib xotira muammoga aylanadi: GPU ning ko'p vaqti hisoblashga emas, KV cache ni xotiradan o'qishga ketadi (memory-bound), hisoblash bloklari bo'sh turadi. VRAM ga sig'masa, ma'lumot sekin RAM ga yoki bir nechta GPU ga bo'linadi. Yechimlar: FlashAttention (xotira chiziqli), GQA/MQA (KV cache kichik), sliding window attention va KV cache kvantlash.

30. Nima uchun "kattaroq model avtomatik ravishda yaxshiroq model degani emas" degan ibora sanoatda haqiqat hisoblanadi?

Qo'shimcha savol: Agar modelni qattiq hardware cheklovlari bo'lgan oddiy qurilmalarda ishlatish kerak bo'lsa, nima uchun muhandislar kichik modellarni ataylab "ortiqcha" o'qitadi (over-train)?

JAVOB

> Kattaroq model avtomatik ravishda yaxshiroq emas, chunki: katta modelga proporsional ravishda ko'p data kerak, aks holda u chala o'qitiladi (Chinchilla 70B o'zidan 4 baravar katta Gopher 280B dan yaxshiroq chiqqan); inference xarajati har bir so'rovda to'lanadi (xotira, tezlik, pul); model qurilmaga sig'ishi kerak; va toza data yoki vazifaga moslashtirilgan kichik model ko'pincha yaxshiroq ishlaydi.
>
> Chinchilla qoidasi faqat training xarajatini hisoblaydi. Telefon yoki noutbukda model millionlab marta ishlatiladi, shuning uchun muhandislar kichik modelni Chinchilla dan ancha ko'p data bilan o'qitadi (over-training). Training bir marta qimmatroq bo'ladi, lekin model kichik, tez va arzon bo'lib qoladi, sifati esa kattaroq modellarga yaqinlashadi. Masalan, Llama 3 8B har bir parametrga taxminan 1,900 token bilan o'qitilgan.